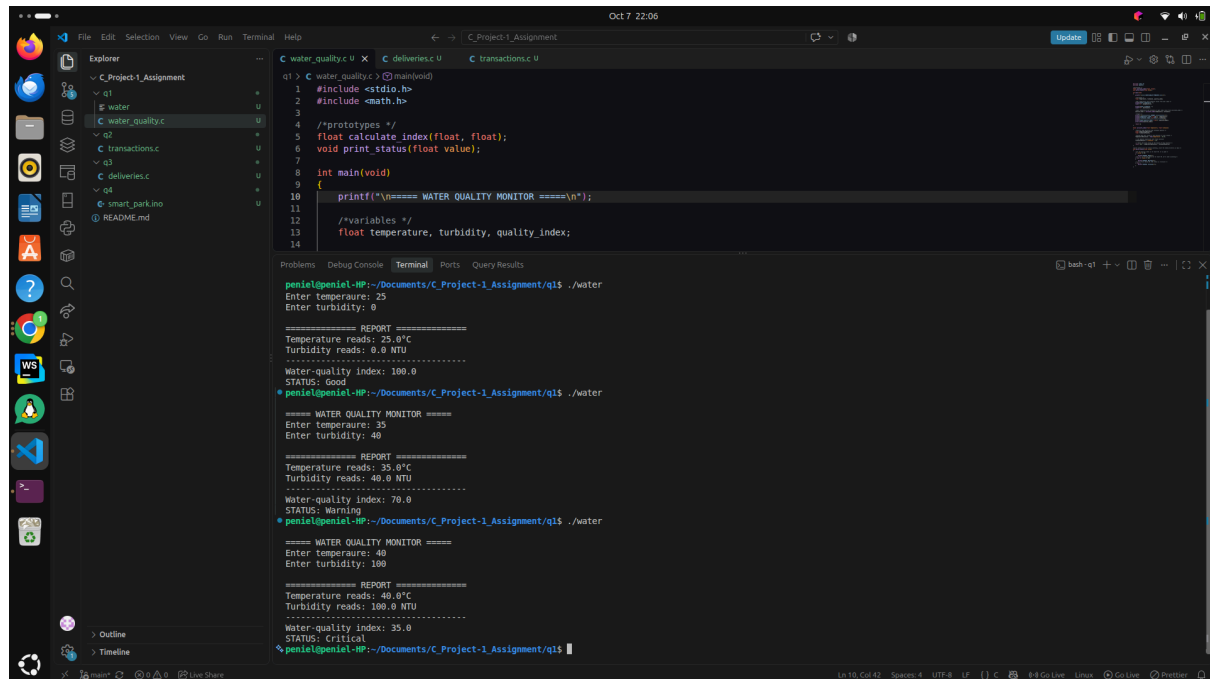


Question 1: Water-Quality Monitor

Source code (water_quality.c)

https://github.com/Peniel-source/C_Project-1_Assignment/blob/main/q1/water_quality.c

Sample output



```
1 #include <stdio.h>
2 #include <math.h>
3
4 //prototypes */
5 float calculate_index(float, float);
6 void print_status(float value);
7
8 int main(void)
9 {
10     printf("\n===== WATER QUALITY MONITOR =====\n");
11     //variables */
12     float temperature, turbidity, quality_index;
13
14     //Test Case 1
15     printf("Enter temperature: ");
16     scanf("%f", &temperature);
17     printf("Enter turbidity: ");
18     scanf("%f", &turbidity);
19
20     quality_index = calculate_index(temperature, turbidity);
21     print_status(quality_index);
22
23     //Test Case 2
24     printf("Enter temperature: ");
25     scanf("%f", &temperature);
26     printf("Enter turbidity: ");
27     scanf("%f", &turbidity);
28
29     quality_index = calculate_index(temperature, turbidity);
30     print_status(quality_index);
31
32     //Test Case 3
33     printf("Enter temperature: ");
34     scanf("%f", &temperature);
35     printf("Enter turbidity: ");
36     scanf("%f", &turbidity);
37
38     quality_index = calculate_index(temperature, turbidity);
39     print_status(quality_index);
40 }
```

```
peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q1$ ./water
Enter temperature: 25
Enter turbidity: 0

===== REPORT =====
Temperature reads: 25.0°C
Turbidity reads: 0.0 NTU
Water-quality index: 100.0
STATUS: Good
peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q1$ ./water

===== WATER QUALITY MONITOR =====
Enter temperature: 35
Enter turbidity: 40

===== REPORT =====
Temperature reads: 35.0°C
Turbidity reads: 40.0 NTU
Water-quality index: 70.0
STATUS: Warning
peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q1$ ./water

===== WATER QUALITY MONITOR =====
Enter temperature: 40
Enter turbidity: 100

===== REPORT =====
Temperature reads: 40.0°C
Turbidity reads: 100.0 NTU
Water-quality index: 35.0
STATUS: Critical
peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q1$
```

a. Real-world application

The use of C in automotive ECUs is extensive. An ECU reads sensor values such as temperature, pressure, and oxygen concentration and makes real-time changes to the fuel injection and ignition system. The properties of C that make it suitable for this application include the ability to control hardware registers directly, the production of small and fast code for small microcontrollers, and timing that is predictable without the influence of garbage collection or heavy runtime.

b. Error analysis

- Syntax error: not using the semicolon at the end of the line `scanf("%f", &temperature);`. This causes the code of C not to follow the rules of grammar and hence results in the compiler showing an error in the program and halting the program from getting executed.
- Semantics error: saying value `> 80` instead of meaning value `>= 80` in the `print_status`. This code works correctly without any complaints, however if the index is exactly equal to 80, it reports it with "Warning" status instead of being OK. Do note that the syntax is still correct but the semantics of the code remains incorrect.
- c. Compilation lifecycle
- Preprocessing stage: the input is `water_quality.c`. The preprocessor puts the contents of `stdio.h` and `Math.h`, replaces macros and deletes comments. The output will be expanded source code in `(.i)` format.

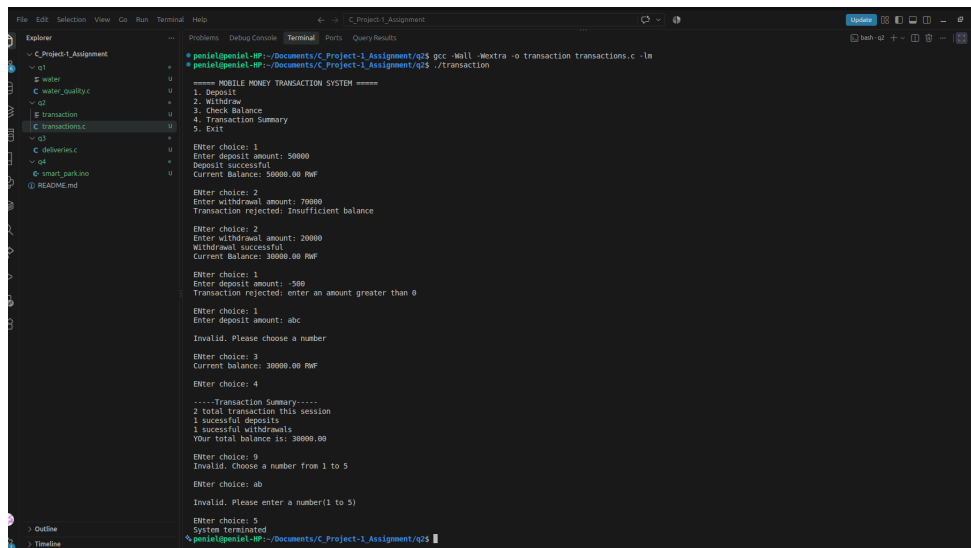
- **Compilation stage:** the input is the .i file. The compiler analyzes the syntax of the code and converts it into assembly code in (.s) format.
- **Assembly stage:** the input for the assembler is .s file. The assembler converts the code into machine code in (.o) format.
- **Linking:** the inputs are the .o file and the libraries it uses (the C standard library and the math library for fabs). The linker combines them and resolves function references such as printf, scanf and fabs. The output is the executable program.

Question 2: Mobile-Money Transaction System

Source code (transactions.c):

https://github.com/Peni-el-source/C_Project-1_Assignment/blob/main/q2/transactions.c

Sample input/output



```

===== MOBILE MONEY TRANSACTION SYSTEM =====
1. Deposit
2. Withdraw
3. Check Balance
4. Transaction Summary
5. Exit

Enter choice: 1
Enter deposit amount: 50000
Deposit successful
Current Balance: 50000.00 RMF

Enter choice: 2
Enter withdrawal amount: 70000
Transaction rejected: Insufficient balance

Enter choice: 2
Enter withdrawal amount: 20000
Withdrawal successful
Current Balance: 30000.00 RMF

Enter choice: 1
Enter deposit amount: .500
Transaction rejected: enter an amount greater than 0

Enter choice: 1
Enter deposit amount: abc
Invalid. Please choose a number

Enter choice: 3
Current balance: 30000.00 RMF

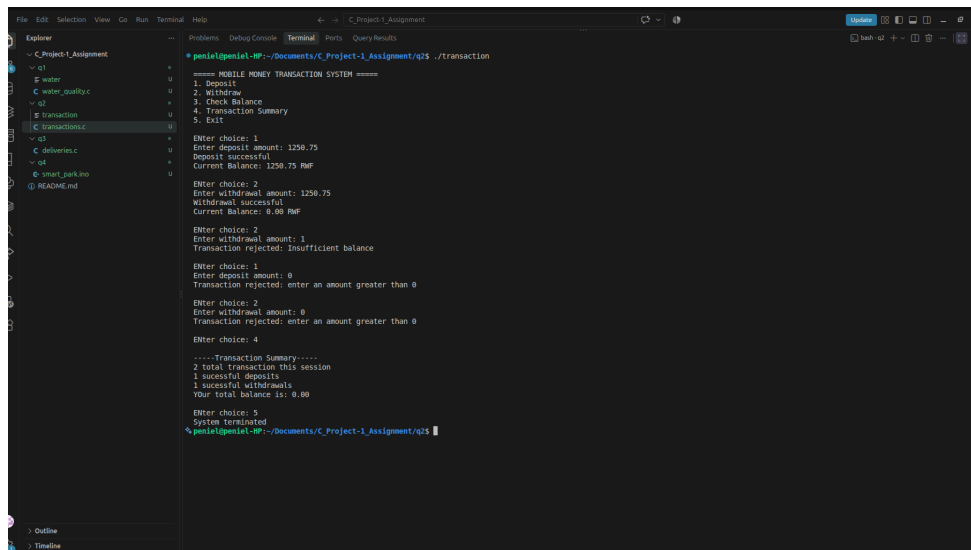
Enter choice: 4
-----Transaction Summary-----
2 total transaction this session
1 successful deposits
1 successful withdrawals
Your total balance is: 30000.00

Enter choice: 5
Invalid. Choose a number from 1 to 5

Enter choice: ab
Invalid. Please enter a number(1 to 5)

Enter choice: 5
System terminated

```



```

===== MOBILE MONEY TRANSACTION SYSTEM =====
1. Deposit
2. Withdraw
3. Check Balance
4. Transaction Summary
5. Exit

Enter choice: 1
Enter deposit amount: 1250.75
Deposit successful
Current Balance: 1250.75 RMF

Enter choice: 2
Enter withdrawal amount: 1250.75
Withdrawal successful
Current Balance: 0.00 RMF

Enter choice: 2
Enter withdrawal amount: 1
Transaction rejected: Insufficient balance

Enter choice: 1
Enter deposit amount: 0
Transaction rejected: enter an amount greater than 0

Enter choice: 2
Enter withdrawal amount: 0
Transaction rejected: enter an amount greater than 0

Enter choice: 4
-----Transaction Summary-----
2 total transaction this session
1 successful deposits
1 successful withdrawals
Your total balance is: 0.00

Enter choice: 5
System terminated

```

Explanation

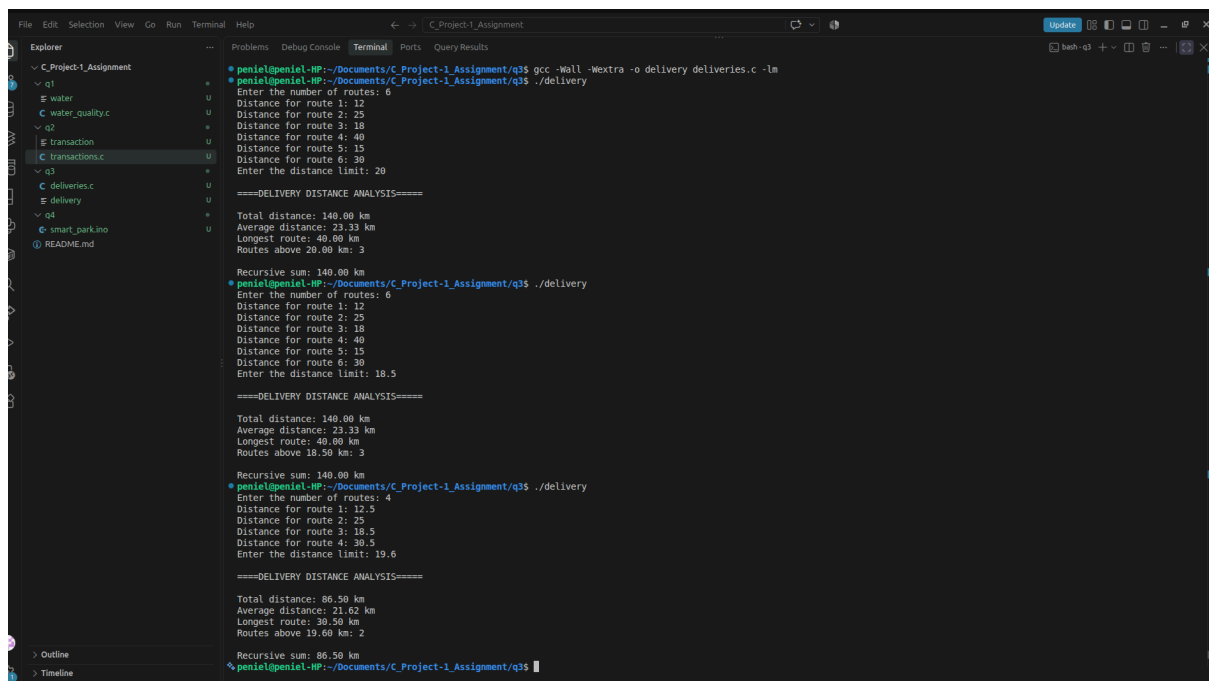
- Conditionals, the transact function works with the help of switch to help in executing deposit, withdrawal, balance inquiry, and transaction summary as well. Any number that is not between 1 to 5 will be handled in default case. Both deposit and withdrawal use if statement for input checks. Input has to be numerical, the amount has to be greater than zero and withdrawal should not be more than available balance.
- Loops, the use of while (1) in main makes the prompt appear as many times as needed for the agent to process the transactions. The show would stop once the agent picks 5.
- Continue, in main, when the input given in menu is not a number then scanf would not return 1, and the error would be printed and the input buffer cleared with clean_buffer, after continuing to next prompt.
- Break, inside transact break is used for the exit of each case in switch and to allow exit from the case without losing the amount due to non-acceptable amounts in input. While in main break exits the while once the agent inputs 5 thereby printing out "System terminated".

Question 3: Delivery Distance Analysis

Source code (deliveries.c)

https://github.com/Peniel-source/C_Project-1_Assignment/blob/main/q3/deliveries.c

Sample input/output



```
peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q3$ gcc -Wall -Wextra -o delivery deliveries.c -lm
peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q3$ ./delivery
Enter the number of routes: 6
Distance for route 1: 12
Distance for route 2: 25
Distance for route 3: 18
Distance for route 4: 40
Distance for route 5: 15
Distance for route 6: 30
Enter the distance limit: 20
=====DELIVERY DISTANCE ANALYSIS=====
Total distance: 140.00 km
Average distance: 23.33 km
Longest route: 40.00 km
Routes above 20.00 km: 3
Recursive sum: 140.00 km
peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q3$ ./delivery
Enter the number of routes: 6
Distance for route 1: 12
Distance for route 2: 25
Distance for route 3: 18
Distance for route 4: 40
Distance for route 5: 15
Distance for route 6: 30
Enter the distance limit: 18.5
=====DELIVERY DISTANCE ANALYSIS=====
Total distance: 140.00 km
Average distance: 23.33 km
Longest route: 40.00 km
Routes above 18.50 km: 3
Recursive sum: 140.00 km
peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q3$ ./delivery
Enter the number of routes: 4
Distance for route 1: 12.5
Distance for route 2: 25
Distance for route 3: 18.5
Distance for route 4: 30.5
Enter the distance limit: 19.6
=====DELIVERY DISTANCE ANALYSIS=====
Total distance: 86.50 km
Average distance: 21.62 km
Longest route: 30.50 km
Routes above 19.60 km: 2
Recursive sum: 86.50 km
peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q3$
```

```

peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q3$ ./delivery
Enter the number of routes: 0
Invalid. number of routes must be between 1 and 100
peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q3$ ./delivery
Enter the number of routes: 101
Invalid. number of routes must be between 1 and 100
peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q3$ ./delivery
Enter the number of routes: abc
Invalid. please enter a number
peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q3$ ./delivery
Enter the number of routes: 2
Distance for route 1: -5
Error. distance must be greater than zero
peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q3$ ./delivery
Enter the number of routes: 2
Distance for route 1: 0
Error. distance must be greater than zero
peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q3$ ./delivery
Enter the number of routes: 2
Distance for route 1: abc
Invalid. Enter a number
peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q3$ ./delivery
Enter the number of routes: 2
Distance for route 1: 10
Distance for route 2: 20
Enter the distance limit: -2
Invalid. Limit cannot be negative.
peniel@peniel-HP:~/Documents/C_Project-1_Assignment/q3$

```

Function break down

The function `find_total` processes the array to find out the amount of distance. The function `calculate_avg` makes use of `find_total` and divides the result by the number of paths. The result is converted into float, so the decimal part is kept. This is an example of function reuse, because one function is embedded in another one. The function `calculate_longest` traverses the array to find the maximum distance. The function `above_limit` finds out the number of routes that are longer than the specified limit.

The function `sum_distance` is recursive. Main function processes the input data, checks its values, recalls each function and creates the report.

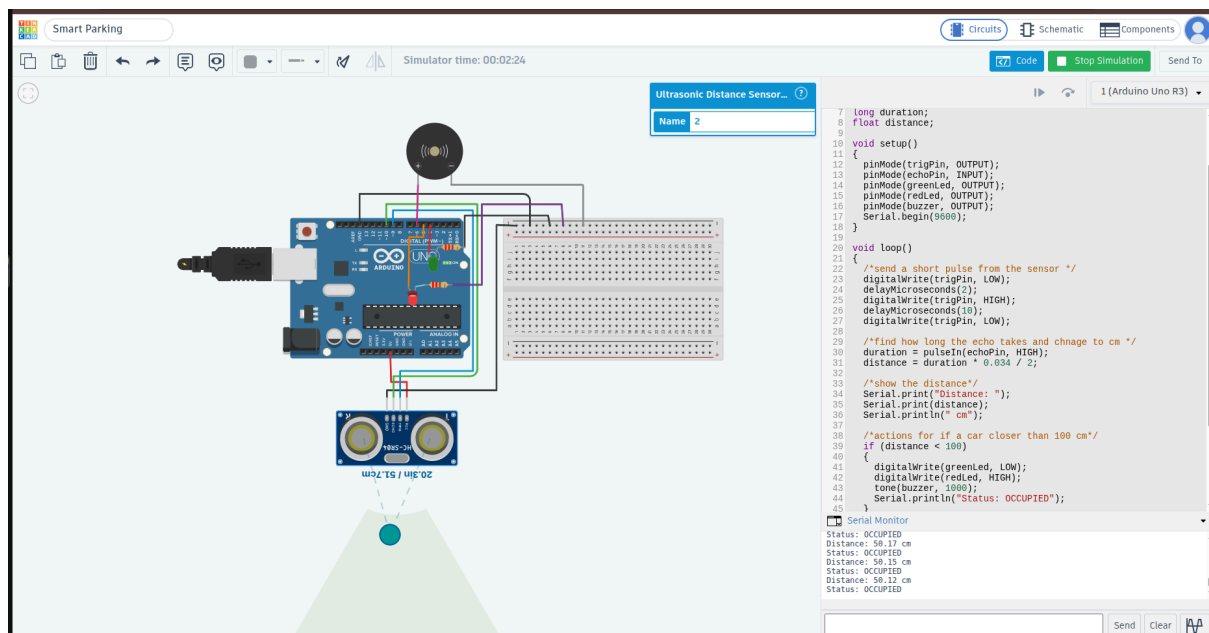
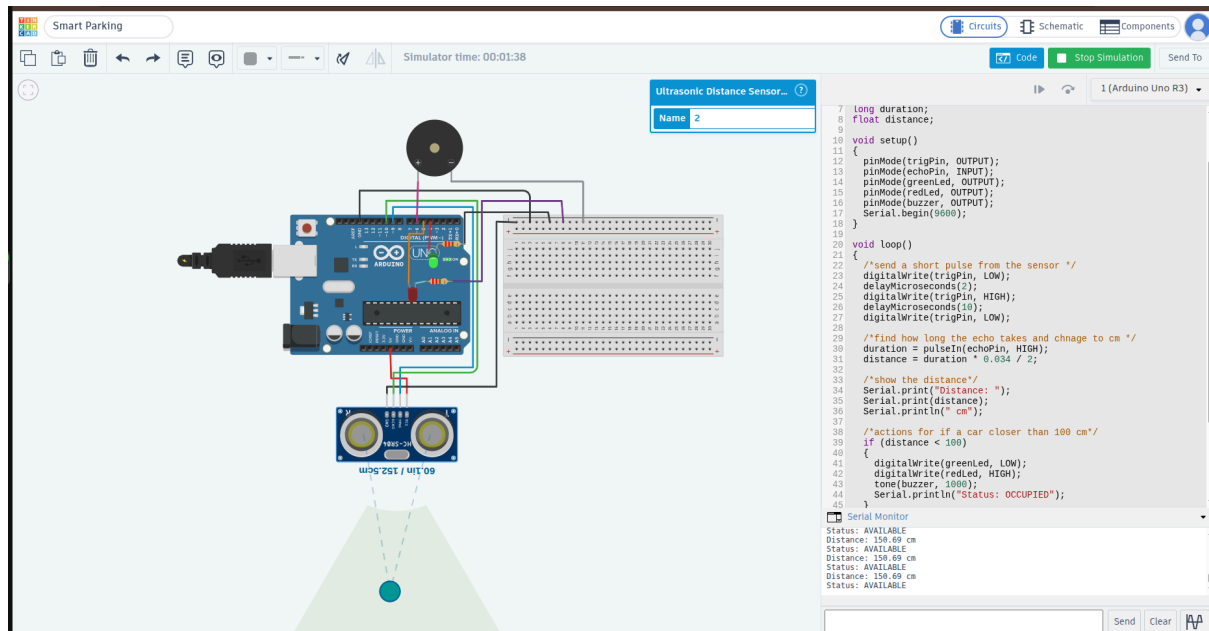
How the recursive function works: `sum_distance(arr, num)` returns the last element of an array plus the total of the first `num - 1` elements. The base case is `num == 0`, which returns 0 because an empty array sums to zero. Every next call changes `num` to a lesser value, enabling the problem to approach the base case. After the base case has been reached, values are returned back through the stream of function calls.

Advantage and limitation of recursion

- **Advantage:** the code is short and mirrors the mathematical definition of a sum (last element plus the sum of the rest), which makes it easy to read.
- **Limitation:** every call uses a stack frame, so memory use grows with the number of routes and a very large array could cause a stack overflow. A simple loop is faster and uses constant memory.

Question 4: Arduino Smart Parking System

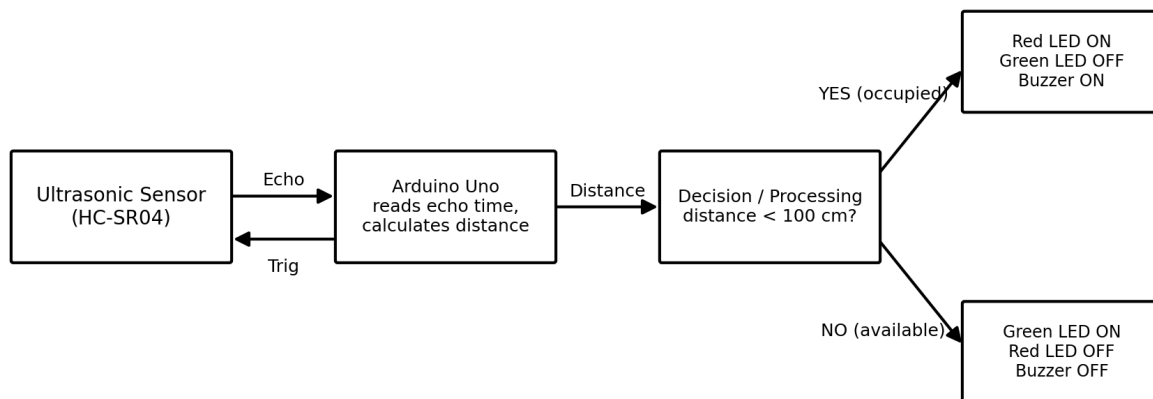
Tinkercad circuit



Connections:

- HC-SR04 Trig to D9, Echo to D10, VCC to 5V, GND to GND
- Green LED anode to D4 through a 220 Ω resistor, cathode to GND
- Red LED anode to D5 through a 220 Ω resistor, cathode to GND
- Buzzer positive to D6, negative to GND

Block diagram



Ultrasonic Sensor → Arduino Uno → Decision/Processing (distance < 100 cm?) → Green LED + Red LED + Buzzer

Arduino source code

https://github.com/Peniel-source/C_Project-1_Assignment/blob/main/q4/smart_park.ino

Simulation test cases

Test 1: vehicle outside the threshold (150 cm). Green LED ON, red LED OFF, buzzer OFF, Serial Monitor shows "Status: AVAILABLE".

Test 2: vehicle within the threshold (50 cm). Red LED ON, green LED OFF, buzzer ON, Serial Monitor shows "Status: OCCUPIED".

Explanation

Role of each component

- **HC-SR04 ultrasonic sensor:** sends out a sound pulse and measures how long the echo takes to return.
- **Arduino Uno:** reads the sensor, calculates the distance, and decides the parking status.
- **Green LED:** shows the parking space is available.
- **Red LED:** shows the parking space is occupied.
- **Buzzer:** sounds an alert when a vehicle is within the threshold.
- **220 Ω resistors:** limit the current through the LEDs.

How sensor data is processed

The Trig pin of the Arduino is pulled high for ten microseconds to produce the pulse signal. Then, the pulseIn command determines the time duration that the Echo pin remains set to high. This time corresponds to the total time for the sound to cover the distance in microseconds. The distance covered (in cm) can thus be calculated using the expression $\text{duration} \times 0.034$ divided by two, since the speed of sound is approximately given by 0.034

cm/ms and this distance is covered twice (in both directions). The output is also broadcast to the Serial Monitor.

How the Arduino controls the outputs

100 cm is chosen as the occupancy threshold for a reasonable distance required to detect a single parking space. If the distance is less than 100 cm, Arduino sets the green LED low, and the red LED high and it triggers the buzzer with `tone(buzzer, 1000)`. In the other case, it sets the green LED high, the red LED low and stops the buzzer with `noTone(buzzer)`. The loop repeats every 200 ms so the output reflects any changes in distance instantly.